

News and Settings including BYOK, Roles, i18n, Stack, and Details



The ideas for setting up a newsfeed or choosing which settings are important or desired for you to set in your app are your own. Changing source and destination on any example or component will make it yours and useful for your projects. For Destination Atlas, we wanted a newsfeed relevant to the selected destination. We wanted to expose enough of BYOK inclusion to inspire thought for how it could apply to your needs.

News

The headlines follow the **selected destination** from Destinations, Map, Globe, or Media; with the same city id as the header.

Viewer sees the results table only. Click a headline to open the publisher article in a new browser tab.

Editor and **Admin** also get search and region filters, plus an **Article detail** panel with summary and a **Read source** link. Admin refreshes the RSS cache under Settings — article 6.

Try it locally:

```
npm run proof:react:live  
# builder API :3000/api + React proof :4311 — swap runtime as needed
```

Open the **News** tab. Pick a destination on **Destinations** or **Settings** to change the selection. With the builder up you get live RSS; when it is offline the app shows cached travel headlines instead of failing.

Destination Atlas

Current and historic information about world locations — Vue proof (DAS-124)

Router
Browser (HTML5 history)
Role Admin
App locale English (English)
Map provider Leaflet
Selected Paris
Theme System

About
Overview
Destinations
Maps
Media
News
Authoring
Plan
Stack
Settings

News

Destination-scoped headlines from Google News RSS via the builder API (~24h cache). Active destination: Paris.

Cached travel news (builder offline) — 4 headline(s). Start live news with `npm run proof:react:live` (or `storybook:react:live` in Storybook).

News search tools

Search and region filters enabled

Region

All regions

News results

4 articles

HEADLINE	SOURCE	REGION	PUBLISHED
Travelling to Paris to see Céline Dion? Here's what you need to know	Canada.com	europe	2026-08-13
Why Paris Official Is Calling Out American Tourists	TravelPulse	europe	2026-06-30
I Fell for a Common Paris Tourist Scam—Here Are the Travel Safety Essentials I Always Carry Now, From \$10	Travel + Leisure	europe	2026-06-26
I asked Paris locals what tourists should skip. Here's what they said	USA Today	europe	2026-05-31

Article detail

Full article summaries

Article detail

Travelling to Paris to see Céline Dion? Here's what you need to know

Canada.com · europe · 2026-08-13

Travelling to Paris to see Céline Dion? Here's what you need to know Canada.com

[Read source](#)

Figure 1: Admin view of news.

Settings

As an app developer, we regularly revisit the same behind-the-scenes challenges. Things such as connecting keys and authentication, enabling views based on roles, ensuring that each language target gets what they asked for, and other minutia.

For this view, we will use the svelte app.

```
npm run proof:svelte
```

Open `http://localhost:4314`.

role, locale, keys, details

role The first setting to choose is the role. Viewer, Admin and Editor roles have different views and accesses to content.

App locale. `domain.i18n.app-language-select` sets a base app language using standard locale codes (`en`, `es`, `fr`, `de`, `ja` in this proof) and emits `locale-change`. You wire that to your own `i18n` — `svelte-i18n`, `vue-i18n`, `react-intl`, `ngx-translate`, whatever you already use.

Keys stay in the browser. Two Admin collapsibles: **Integration keys** and **AI providers** — both bring-your-own-key (BYOK) vaults. Maps can use Google or MapTiler. AI is OpenAI, Anthropic, Gemini, Azure, Ollama — all on the same Settings page. The vault is encrypted in this browser. Nothing goes to a RosettaDash server. Map reads a stored key when one exists; otherwise it falls back to what the proof ships in `env`. Viewer and Editor can see that the vault exists. Only Admin can write it.

News feeds (admin). A third Admin collapsible on Settings — next to Integration keys and AI providers. Only **Admin** can refresh the builder's Google News RSS cache (~24 hours). Viewer and Editor see the gate message; they browse headlines on the **News** tab without admin tools.

The panel shows article count, last fetch time, and configured feed queries. **Refresh news cache** calls `POST /api/news/admin/refresh` through the same `/builder-api` proxy the proof apps use.

Try it locally:

```
npm run proof:react:live
```

Open **Settings** as Admin (`?role=admin` in the URL). Expand **News feeds (admin)** and refresh. Then open **News** with a destination selected to see updated headlines.

React, Vue, Svelte, and Web Components proofs ship this panel. Angular proof has the News tab but not the Settings admin block yet.

Destination Atlas

Current and historic information about world locations — Svelte proof (DAS-158)

Router
Browser (HTML5 history)
Role
Admin
App locale
English (English)
Map provider
Leaflet
Selected
Tokyo
Theme
System

About
Overview
Destinations
Maps
Media
Authoring
Plan
Stack
Settings

Settings

Theme
System

Role
Admin
Preview permissions for gated screens.

App locale
English
Base locale for developer-owned destination labels.

Map provider
Leaflet
Default 2D map engine for the Map screen.

Selected
Tokyo
Active destination across tabs and URL state.

Integration keys (BYOK)
Google Maps, MapTiler, News API

AI providers (BYOK)
OpenAI, Anthropic, Gemini, Azure, Ollama

Feedback
This is a demonstration, and feedback is not functional here.
Submit feedback

Component source

Svelte components, props, and nested structure for this screen.

```

<SettingsScreen>
  <header><h2>Settings</h2><ThemeToggle /></header>
  <AtlasContextControls highlightField={...}>
    <AppLanguageSelect locales={DEFAULT_APP_LOCALES} value={locale} onLocaleChange={...} />
  </AtlasContextControls>

```

Figure 2: Settings — locale, BYOK vaults, and the Admin news-feed refresh panel.

Plan — roles

Only Editor and Admin have access to the Plan tab. This is the roles gate in action. Here, if you have permissions, you can invite a person, assign a role, name the trip, pick a city, set dates, duration, travelers, and keep notes. Users without permissions (i.e., viewers) are rerouted to the About landing page.

Destination Atlas

Current and historic information about world locations — Svelte proof (DAS-158)

Router
Browser (HTML5 history)
Role Editor
App locale English (English)
Map provider Leaflet
Selected Tokyo
Theme System

About
Overview
Destinations
Maps
Media
Authoring
Plan
Settings

Plan

Trip planning demo — forms, invites, and timers behind role gates.

Trip editor

Trip editor unlocked

Trip name

Spring getaway

Primary destination

Tokyo

Trip dates

04/10/2026 to 04/17/2026

Trip duration (days)

8

Share itinerary with team

Notes

Packing list, reservations...

Invite team member

name@company.com

Assign role

Booking countdown

0 ticks

Send invite

Confirm access

Component source

Svelte components, props, and nested structure for this screen.

```

<PlanScreen userRole={userRole}>
  <RoleGate currentRole={userRole} allowedRoles={['editor', 'admin']} label="Trip editor">
    ~ trip planner form ~
  </RoleGate>
</PlanScreen>

```

Figure 3: Plan — trip editor for Editor and Admin; the gate’s hidden status for Viewer.

Stack — invisible infrastructure

Stack is Admin only. A read-only grid of the infra nodes from the builder article: env keys, Postgres, Mongo, MySQL, Supabase, Nest, Express, Next, Nuxt. Choosing a server or a database in the builder generates stubs and .env templates. Connection strings stay in env vars, not in the exported source.

On the **React** and **Angular** proofs, the Stack tab can go one step further: it loads seeded orders rows from a generated API backed by PostgreSQL in Docker — the same path export gives you (UI → server → database). The infra grid above still applies; the live table shows that path working, not just the export-wizard labels.

Try it locally — one command prepares the parity stack and opens the React proof (Angular uses Nest on :53101 instead of Next on :53103):

```

npm run parity:stack:proof:react
# or: npm run parity:stack:proof:angular

```

Open Stack as **Admin**. A green banner and a **Seeded orders** table mean the export is serving rows. If Docker is not up, the tab explains what to run instead of erroring out.

This article uses the **Svelte** proof (npm run proof:svelte, port 4314) for Settings and Plan. Svelte Stack shows the infra grid only — use React or Angular when you want the live orders demo. From the cloned repo root, parity:db:up seeds four databases in Docker;

parity:generate and parity:servers:up boot generated Nest, Express, Next, and Nuxt apps that serve the same orders rows the builder previews (ports 53101–53104).

Destination Atlas

Current and historic information about world locations — Svelte proof (DAS-158)

Router

Browser (HTML5 history)

Role Admin

App locale English (English)

Map provider Leaflet

Selected Tokyo

Theme System

About

Overview

Destinations

Maps

Media

Authoring

Plan

Stack

Settings

Stack

Read-only infra configuration demo for export wizard nodes. Integration keys reflect BYOK status from Settings.

Infrastructure stack

Admin infrastructure panel

INFRA

Environment config

DATABASE_URL, GOOGLE_MAPS_KEY, NEWS_API_KEY, FEATURE_FLAGS

INFRA

Sessions

MONGODB_URI sessions

INFRA

API (Nest)

globalPrefix: api

INFRA

Web (Nuxt)

Key status: DATABASE_URL (not set)GOOGLE_MAPS_KEY (not set)NEWS_API_KEY (not set)FEATURE_FLAGS (not set)

INFRA

Legacy CRM

MYSQL_URL contacts

INFRA

API (Express)

globalPrefix: api

INFRA

Analytics DB

DATABASE_URL destinations

INFRA

Supabase

SUPABASE_URL profiles

INFRA

Web (Next.js)

Component source

Svelte components, props, and nested structure for this screen.

```
<StackScreen userRole={userRole}>
  <RoleGate currentRole={userRole} allowedRoles={['admin']} label="Infrastructure stack">
    <EnvConfig envKeys="DATABASE_URL, GOOGLE_MAPS_KEY, NEWS_API_KEY" keyStatus={...} />
  </RoleGate>
</StackScreen>
```

Figure 4: Stack — Admin-only grid of invisible infra. Env row reflects the vault.

7

Destination Atlas

Current and historic information about world locations — Svelte proof (DAS-158)

Router
Browser (HTML5 history)
Role
Viewer
App locale
English (English)
Map provider
Leaflet
Selected
Tokyo
Theme
System

About
Overview
Destinations
Maps
Media
Settings

Settings

Theme
System

Role
Viewer
Preview permissions for gated screens.

App locale
English
Base locale for developer-owned destination labels.

Map provider
Leaflet
Default 2D map engine for the Map screen.

Selected
Tokyo
Active destination across tabs and URL state.

Integration keys (BYOK)
Google Maps, MapTiler, News API

AI providers (BYOK)
OpenAI, Anthropic, Gemini, Azure, Ollama

Feedback
This is a demonstration, and feedback is not functional here.
Submit feedback

Component source

Svelte components, props, and nested structure for this screen.

```

<SettingsScreen>
  <header><h2>Settings</h2><ThemeToggle /></header>
  <AtlasContextControls highlightField={...}>
    <AppLanguageSelect locales={DEFAULT_APP_LOCALES} value={locale} onLocaleChange={...} />
  </AtlasContextControls>

```

Figure 5: Optional detail — a key marked configured after save, or a Viewer staring at a locked vault.

Next

The final article ties it all together and digs into how one canvas was used to generate five runtimes, with one of them mixing frameworks on purpose.